

# Learning to Herd Under Limited Visibility: Recurrent PPO with Procedural Curricula for Single-Agent Shepherd

Author One

Department of Computer Science  
University Name  
City, Country  
author.one@university.edu

Author Two

Department of Computer Science  
University Name  
City, Country  
author.two@university.edu

Author Three

Department of Computer Science  
University Name  
City, Country  
author.three@university.edu

**Abstract**—Shepherding asks one agent to move a flock of reactive, non-cooperative agents into a goal region. Collect-and-drive heuristics solve the open-field version well but degrade when sensing is local, obstacles are present, or the flock splits. We study whether a learned controller can close that gap without overfitting to its training layouts. A simulated dog with a finite visibility radius controls a Strömbom-style flock of 6 to 16 sheep. The dog observes an egocentric, normalized state with masked sheep slots, a lidar scan, and a short memory of the last seen flock centroid. We train a recurrent PPO policy with a geometric shaped reward, procedurally generated obstacle topologies, domain randomization over flock dynamics and sensing, and an adaptive curriculum with hysteresis. Evaluation uses 150 held-out episodes per scenario with bootstrap confidence intervals, and compares against a cluster-aware collect-and-drive heuristic and a random-forest clone of it. An earlier policy trained on a fixed set of layouts reached 75% in-distribution success but 0% on held-out split-field and open-field maps. The procedurally trained policy reaches 45% and 49% success on those maps and beats the heuristic on the split field (58% versus 32% of the flock delivered). The heuristic remains stronger on the procedural test suite (47% versus 26%) and on the open field (99% versus 57%), and no agent solves corridor or narrow-gate maps. We also report the curriculum failure modes found during training and the fixes that removed them.

**Index Terms**—Shepherding, Reinforcement learning, Partial observability, Curriculum learning, Domain randomization, Generalization

## I. INTRODUCTION

Shepherding is the problem of steering a group of passive, interacting agents into a target region using one or a few active agents [1], [2]. The herder never carries a sheep. It can only change where the flock wants to go by choosing where to stand. The same structure appears in livestock robotics [3], crowd guidance, and the indirect control of robot swarms [4], [5].

Strömbom *et al.* showed that a short program reproduces the GPS traces of working sheepdogs: collect the furthest sheep when the flock is too spread out, otherwise drive the flock from behind toward the goal [2]. That rule assumes the dog can see the whole flock and that the field is mostly open. Both assumptions fail when visibility is limited, when obstacles block the direct route, or when the flock splits into separate groups. Tuning the rule for each geometry is

tedious, which makes learning a policy attractive. The learning problem is hard, though. It is a partially observable Markov decision process (POMDP) with a long horizon, a many-body plant, and a sparse success condition.

Our first attempt illustrates the main risk. A recurrent PPO policy trained on four fixed obstacle layouts, with the flock always spawned opposite the goal, succeeded in 75% of training episodes but in none of the held-out split-field or open-field episodes, where the heuristic succeeded in 25% and 100% (small evaluation budget, single seed). The policy had learned the layouts rather than how to herd, a failure mode well documented for RL benchmarks with a fixed set of levels [6]. This paper describes the redesign that followed and measures how far it goes.

Our contributions are:

- A herding environment with procedurally generated obstacle topologies, randomized flock size, dynamics, sensing noise and goals, and a held-out test distribution with wider parameter ranges than training (Section III).
- A recurrent PPO agent with an egocentric observation, a geometric shaped reward, and an adaptive curriculum. We document four failure modes of the curriculum and checkpointing pipeline and the fixes for each (Section IV, Section VI.C).
- A comparison with a collect-and-drive heuristic and a behavioral-cloning baseline on 900 held-out episodes per agent, with bootstrap confidence intervals (Section VI).

The learned policy transfers to held-out geometry that defeated the earlier version, and beats the heuristic when the flock must be herded around bars that split it. It does not yet match the heuristic on the full procedural test suite, and no method solves maps that require threading the flock through a passage.

## II. RELATED WORK

*Heuristic shepherding.* Reynolds’ boids established local attraction, alignment and repulsion as a model of flocking [7]. Lien *et al.* described geometric herding primitives for motion planning [1], and Vaughan *et al.* herded real ducks with a mobile robot [4]. The collect-and-drive algorithm of

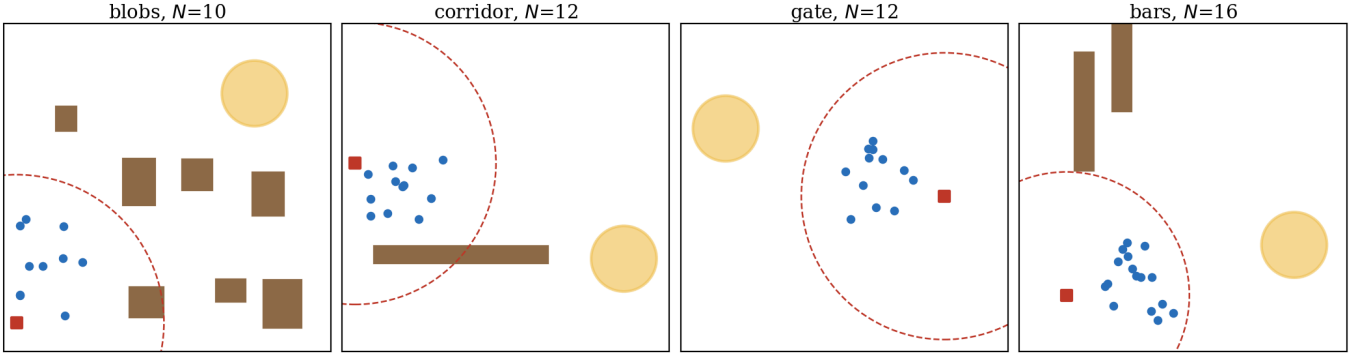


Fig. 1. Samples from the held-out procedural test distribution, one per obstacle topology. Red square: dog. Dashed circle: its visibility radius. Blue: sheep. Yellow disc: goal region.  $N$  is the flock size.

Strömbom *et al.* [2] and its robotic extensions [8] are the standard single-herder baseline. Pierson and Schwager analyze herding of non-cooperative agents from a control perspective [5], and King *et al.* review robot herding inspired by biology [3].

*Learning to shepherd.* Hussein *et al.* learn the collect and drive behaviors with curriculum PPO instead of a waypoint generator [9]. Hasan *et al.* treat cooperative multi-shepherd herding as payload protection [10], and Napolitano *et al.* learn decentralized policies for several herders and non-cohesive targets [11]. These works usually assume wider sensing than ours, or split the task into target assignment and single-target driving. We learn one continuous steering command for a single dog that must remember sheep it can no longer see and route around obstacles.

*Generalization and evaluation.* Recurrent policies are the usual response to partial observability [12], [13]. Domain randomization [14] and curricula [15] widen the training distribution, and procedurally generated levels with a held-out test set are the standard way to measure whether a policy generalizes [6]. Agarwal *et al.* show that RL results from few runs and few episodes are often not reliable [16]. We therefore report bootstrap intervals [17] and flag single-seed results. Behavioral cloning [18], [19] is a common imitation baseline, and its compounding error under distribution shift is well known [20].

### III. ENVIRONMENT

#### A. Workspace and flock

The workspace is the square  $[0, 20]^2$ . A dog at position  $d_t$  herds  $N$  sheep  $s_t^i$  toward a goal  $g$  among at most eight axis-aligned rectangular obstacles. The action  $a_t \in [-1, 1]^2$  is normalized to unit length, so the dog always moves one unit per step, and is clipped to the longest collision-free prefix of the step. An episode succeeds when every sheep is within  $\rho = 2$  of  $g$  and is truncated after 700 steps.

Each sheep moves at a constant speed  $v_s$  in the direction of a sum of forces, following [2], [7]. It flees the dog with unit weight when the dog is within the flee radius  $R_f$  and is pulled toward the flock centroid with gain  $\alpha$ . It is repelled

by neighbors closer than a distance  $R_r$  and pulled toward a leader, the sheep nearest the centroid, with gain  $\beta$ . A linear repulsion pushes it away from obstacles within a threshold distance, and Gaussian noise with standard deviation 0.02 is added. While a sheep is inside the flee radius it also receives a small pull of 0.03 toward the goal. The nominal values are  $v_s = 0.32$ ,  $R_f = 5.5$ ,  $\alpha = 0.07$ ,  $R_r = 1.0$  and  $\beta = 0.04$ .

#### B. Observation

The dog sees sheep within a Euclidean visibility radius  $R_v$  (nominally 7.5). Obstacles do not block sight. The policy receives a 98-dimensional egocentric vector with no absolute position. It contains the unit direction and scaled distance to the goal, the distance to each arena wall along the two axes, and the previous action. It also has a summary of the visible sheep: visible fraction and count, and the centroid offset, mean and maximum distance, and spread, all scaled by  $R_v$ . A memory block holds the offset to the last seen flock centroid, the time since the flock was last seen, and the direction and distance from that centroid to the goal. A time-remaining feature follows. Next come 16 sheep slots of (relative  $x$ , relative  $y$ , visible flag), with invisible sheep and unused slots set to zero. The last 24 values are lidar ranges to obstacles and arena walls, up to 15 units. Sheep positions carry Gaussian observation noise, and executed actions carry Gaussian action noise. The fixed slot count lets one network handle flocks of 6 to 16 sheep.

#### C. Scenarios and splits

Layouts are drawn from five topologies: *open* (no obstacles), *blobs* (scattered rectangles), *corridor* (parallel walls), *gate* (a wall across the field with one opening) and *bars* (short barriers that split the flock). Fig. 1 shows one sample of each obstacle topology. Goals are uniform in  $[2.4, 17.6]^2$ , and the flock spawns in a small box at least 9 units from the goal with the dog behind it. Obstacles near the spawn or the goal are removed, and a flood fill rejects layouts in which the goal is unreachable.

We use three splits. *Train* samples topologies, dynamics and noise from the training ranges, scaled by the curriculum (Section IV.C). *Validation* uses the same ranges at full breadth with a separate seed stream, and selects the checkpoint. *Test*

(*test\_procedural*) uses wider ranges that extend past the training ranges at both ends, on seeds not used in training or validation. Its flocks have 8 to 16 sheep instead of 6 to 14. It also has more obstacle blobs, narrower gates (2.2 to 4.0 units instead of 3.0 to 6.0), longer walls, and visibility radii from 4.5 to 10.0 instead of 5.0 to 9.5. The test ranges for sheep speed, cohesion, repulsion, flee radius and noise are wider in the same way.

Five hand-authored presets serve as further held-out maps with fixed parameters and 10 sheep. In *corridor* the flock must pass two staggered pairs of walls. *Dense* has five larger blobs near the start, and *narrow gate* has two offset walls with openings. *Split field* has three horizontal bars between the spawn and the goal. *Open field* has no obstacles, a small visibility radius (5.6) and faster sheep. None of these presets was used for training or model selection.

## IV. METHOD

### A. Reward

The per-step reward is a sum of geometric terms. Let  $c_t$  be the flock centroid,  $D = 20\sqrt{2}$  the workspace diagonal,  $H_t$  the area of the flock’s convex hull,  $\delta_t$  and  $m_t$  the mean and maximum sheep-to-goal distance, and  $\nu_t$  the visible fraction. The base term rewards a centroid near the goal and a compact flock, penalizes a dog standing between the flock and the goal, and rewards staying near the flock:

$$r_t^{\text{base}} = \left(1 - \frac{\|c_t - g\|}{D}\right) - 0.1 \frac{H_t}{400} - 0.5 \mathbb{1}[\|d_t - g\| < \|c_t - g\|] + 0.3 \left(1 - \frac{\|d_t - c_t\|}{D}\right)^{(1)}$$

The full reward adds progress, visibility, flock-cohesion, positioning and collision terms:

$$r_t = r_t^{\text{base}} + 3.0(\delta_{t-1} - \delta_t) + 2.5(m_{t-1} - m_t) + 0.8(\nu_t - \nu_{t-1}) - 0.05(1 - \nu_t) - 0.25 \mathbb{1}[\nu_t = 0] - 0.05 n_t^{\text{stray}} + 0.35 \left(1 - \frac{\|d_t - p_t\|}{D}\right) + r_t^{\text{coll}}, \quad (2)$$

where  $n_t^{\text{stray}}$  counts sheep more than  $1.8\rho$  from the centroid. The drive point  $p_t$  lies behind the centroid on the goal axis at distance  $\min(0.7R_v, 0.9R_f)$ , which is where a collect-and-drive dog would stand [2]. The collision term is  $-0.2$  when the dog first touches an obstacle or wall and  $-0.03$  per step while contact lasts. Success adds a terminal bonus of  $+20$ . We lowered this bonus from an earlier  $+125$  because, with reward normalization, a bonus that large dominated the value targets. The progress terms are differences of potentials [21].

### B. Recurrent PPO

The policy is recurrent PPO [22] with an LSTM actor-critic of hidden size 256 (`MlpLstmPolicy` from `sb3-contrib` [23]). We collect 256 steps from each of 12 parallel environments per update, then train for 10 epochs with batch size 384,  $\gamma = 0.99$ , GAE  $\lambda = 0.95$ , clip range 0.2 and entropy coefficient 0.005. The learning rate anneals linearly from  $3 \times 10^{-4}$  to zero over  $6 \times 10^5$  steps. Observations and rewards are

normalized with running statistics. Every 25k steps the policy is evaluated on 30 validation episodes, and the checkpoint with the highest mean fraction of the flock at the goal is kept together with its normalization statistics. Training takes about 75 minutes on a 16-core CPU.

### C. Adaptive curriculum

The curriculum has four stages,  $\sigma \in \{0, 0.33, 0.66, 1\}$ . Stage 0 samples open, blob and bar layouts, and shrinks every randomized range to 25% of its width around the nominal value. Stage 0.33 adds gates and widens the ranges to 60%. Stage 0.66 adds corridors, uses the full ranges, and randomizes the flock size. Stage 1 uses the same distribution as stage 0.66 but has stricter gates. The goal is random at every stage.

A callback tracks a window of the last 40 training episodes. It promotes the agent when the window meets the next stage’s thresholds on the fraction of the flock at the goal, visibility, collision events and progress reward, after a minimum share of the training budget. The thresholds for stages 0.33, 0.66 and 1 require at least 30%, 55% and 70% of the flock at the goal, and at most 22, 19 and 16 collision events per episode. Two rules prevent oscillation. After any stage change, the stage is fixed for 25k steps. A demotion also requires missing a threshold by 25% of its value. Collision counts can block a promotion but never cause a demotion; Section VI.C explains why.

### D. Baselines

*Heuristic.* The expert is a collect-and-drive controller in the style of [2]. When at least four sheep are visible and the two furthest apart are more than 2.0 units apart, it splits the visible sheep into two clusters and works on the one further from the goal. If a sheep in that cluster is more than 3.2 units from the cluster centroid, the dog moves behind that sheep to collect it. Otherwise it moves to a drive point behind the centroid. An obstacle-avoidance term is added to the command. With no sheep visible, it returns to the last seen centroid or searches toward the goal. The heuristic has an advantage over the learned agents: it reads the absolute dog position and every obstacle rectangle regardless of visibility. It also uses the nominal visibility and flee radii rather than the randomized ones.

*Behavioral cloning (BC).* A random forest [24] with 300 trees and depth 18 regresses the expert’s action from raw coordinates plus 15 geometric summaries of the visible flock. On an episode-level hold-out it reaches  $R^2$  of 0.59 and 0.69 on the two action components, with a mean heading error of  $29^\circ$ . The most important feature is the fraction of visible sheep in the focus cluster (impurity importance 0.147), which is the quantity the expert uses to detect a split. The forest was trained on demonstrations from an earlier version of the environment, in which goals were confined to the far quadrant. Those demonstrations are no longer available, so we could not retrain it on the procedural distribution. Its results below therefore test a clone trained on a narrower distribution, not cloning in general.

TABLE I

HELD-OUT RESULTS. FRACGOAL WITH 95% BOOTSTRAP INTERVAL, SUCCESS RATE (SR) AND FINAL MEAN SHEEP-TO-GOAL DISTANCE (DIST.). ALL AGENTS SEE THE SAME EPISODES WITH A 10-SHEEP FLOCK. WITH THE RANDOMIZED 8 TO 16 SHEEP FLOCK, RECURRENT PPO SCORES 0.209 [0.155, 0.264], SR 0.12, ON THE PROCEDURAL TEST. BEST FRACGOAL PER ROW IN BOLD WHEN INTERVALS DO NOT OVERLAP.

| Scenario        | Heuristic                   |      |       | Behavioral cloning   |      |       | Recurrent PPO (ours)        |      |       |
|-----------------|-----------------------------|------|-------|----------------------|------|-------|-----------------------------|------|-------|
|                 | FracGoal                    | SR   | Dist. | FracGoal             | SR   | Dist. | FracGoal                    | SR   | Dist. |
| Procedural test | <b>0.468</b> [0.395, 0.537] | 0.35 | 3.44  | 0.010 [0.001, 0.024] | 0.00 | 12.20 | 0.261 [0.200, 0.322]        | 0.16 | 4.28  |
| Split field     | 0.317 [0.276, 0.360]        | 0.08 | 3.97  | 0.196 [0.161, 0.233] | 0.01 | 3.86  | <b>0.576</b> [0.512, 0.644] | 0.45 | 2.21  |
| Dense           | 0.269 [0.207, 0.333]        | 0.19 | 6.41  | 0.062 [0.036, 0.090] | 0.00 | 7.83  | 0.289 [0.236, 0.345]        | 0.11 | 3.85  |
| Open field      | <b>0.989</b> [0.975, 1.000] | 0.98 | 1.21  | 0.599 [0.519, 0.674] | 0.39 | 2.17  | 0.565 [0.490, 0.637]        | 0.49 | 3.09  |
| Corridor        | 0.000 [0.000, 0.000]        | 0.00 | 12.21 | 0.008 [0.000, 0.023] | 0.00 | 10.27 | 0.007 [0.000, 0.017]        | 0.00 | 11.58 |
| Narrow gate     | 0.003 [0.000, 0.007]        | 0.00 | 6.75  | 0.021 [0.008, 0.036] | 0.00 | 5.36  | 0.005 [0.000, 0.015]        | 0.00 | 10.68 |

## V. EXPERIMENTAL SETUP

Each agent is evaluated on the procedural test split and the five presets, 150 episodes per scenario for the heuristic and recurrent PPO and 100 for BC. All agents use deterministic actions and see the same episodes: seeds start at 100000 for every agent, so layouts, goals and dynamics match. The heuristic and BC read the older observation format, which has exactly ten sheep slots. For a matched comparison we therefore also fix the flock at ten sheep for recurrent PPO; we report the randomized-flock result separately.

The main metric is the fraction of the flock inside the goal region at the end of the episode (FracGoal). It gives partial credit, which matters when most episodes end with a few stragglers outside the goal. We also report success rate (SR, all sheep inside) and the mean sheep-to-goal distance. The 95% intervals for FracGoal are percentile bootstrap intervals over episodes with 2000 resamples [17]. The recurrent PPO results come from one training seed; runs with further seeds are in progress.

## VI. RESULTS

### A. Held-out comparison

Table I and Fig. 2 give the results. On the procedural test split the heuristic delivers 47% of the flock and succeeds in 35% of episodes. Recurrent PPO delivers 26% and succeeds in 16%, and the intervals do not overlap. With its randomized flock sizes the test is harder still, and FracGoal drops to 21%.

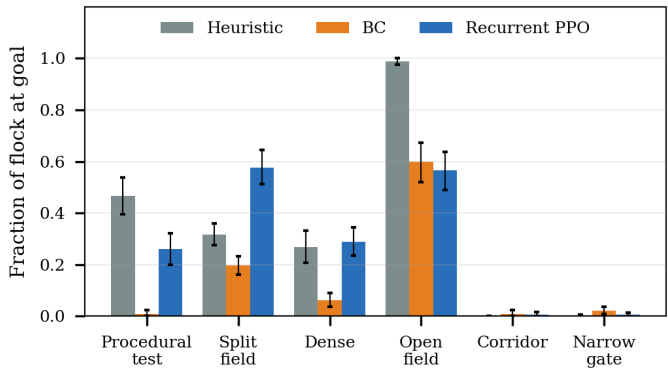


Fig. 2. Fraction of the flock at the goal on the held-out scenarios. Error bars are 95% bootstrap intervals over episodes.

The presets give a more varied picture. On the split field, recurrent PPO delivers 58% of the flock and succeeds in 45% of episodes, against 32% and 8% for the heuristic. It also ends closer to the goal (2.21 versus 3.97). The bars tend to break the flock into groups on different sides of an obstacle, and the heuristic works on one cluster at a time. In 92% of its split-field episodes it runs out of time before the whole flock reaches the goal (mean episode length 660 of 700 steps). On the dense map the two agents deliver the same fraction within noise, although the heuristic finishes more episodes (19% versus 11%) and the learned policy leaves the flock closer to the goal on average. On the open field the heuristic is nearly perfect (99%, finishing in 191 steps on average), while recurrent PPO delivers 57% and succeeds in 49% of episodes. This is the setting that Strömbom’s rule was designed for, and the learned policy does not recover it fully. Corridor and narrow-gate maps defeat all three agents. Both require the dog to push the whole flock through an opening, and neither the reward nor the heuristic has a term for that.

Compared with the earlier structured policy, which succeeded in none of the held-out split-field and open-field episodes, the procedurally trained policy succeeds in about half of them. That earlier result used a small evaluation budget, so the comparison is qualitative, but the size of the change is well beyond evaluation noise.

The clone does worst in nearly every scenario. It keeps some competence on the open field (60% delivered), the

setting closest to its training data, and falls to 1% on the procedural test. Its 29° offline heading error compounds in closed loop [20], and the change in goal distribution puts most test states outside its training data.

### B. Training dynamics

Fig. 3 shows validation performance and the curriculum stage for the reported run. Validation FracGoal rose from 0.13 at 25k steps to 0.41 at 600k, with most of the gain in the second half as the learning rate decayed. The final checkpoint was also the best one. Collision events per training episode fell from about 36 to between 14 and 18, although collisions were never a reason for demotion.

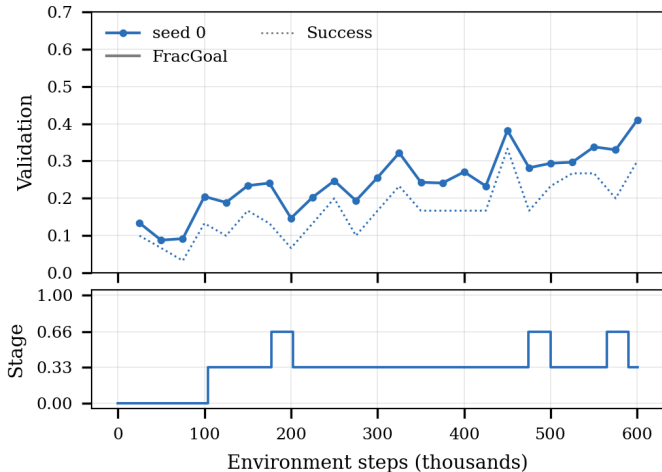


Fig. 3. Validation FracGoal (solid) and success rate (dotted) every 25k steps, 30 episodes each, and the curriculum stage (bottom).

The curriculum reached stage 0.33 at 104k steps and stage 0.66 at 178k, then alternated between the two. The agent spent 17% of training at stage 0, 70% at stage 0.33 and 13% at stage 0.66, and never reached stage 1. Each demotion from 0.66 came from delivery: at full breadth the agent could not keep FracGoal above the demotion bound of 0.41. Most learning therefore happened at 60% breadth, without corridors or randomized flock sizes. That is consistent with the weak corridor results and with the gap between the fixed and randomized flock results on the test split.

### C. Training stability

Four earlier runs with the same seed failed or were stopped, each for a different reason (Table II). The first run pinned the goal to one corner in stage 0. Validation always randomizes the goal, so validation FracGoal was exactly zero at all 17 checkpoints up to 433k steps. Without hysteresis, the stage also flipped between 0 and 0.33 within a single rollout. After goals were randomized from stage 0 and hysteresis was added, the collision threshold became the only binding constraint. Arena walls count as collisions, and random goals often lie near a wall, so a competent policy settled at 17 to 18 collision events per episode, above the demotion bound. Raising the thresholds exposed a second problem: each promotion adds harder layouts, which raised collisions enough to trigger a demotion as soon as the 25k-step lock expired, even while

delivery improved. Removing collisions from the demotion rule fixed this. Collisions then fell below the old thresholds on their own, because the reward still penalizes them. The fourth run exposed a checkpointing bug: the best checkpoint was saved without the observation-normalization statistics of that moment, so it could only be evaluated with mismatched normalization. With 12 validation episodes, the standard error of FracGoal was about 0.1, so selecting the best checkpoint mostly picked the luckiest draw. The reported run pairs each checkpoint with its statistics, uses 30 validation episodes, and anneals the learning rate.

TABLE II  
FAILURE MODES FOUND IN SUCCESSIVE TRAINING RUNS AND THE CHANGE THAT REMOVED EACH.

| Run | Symptom                                                                   | Fix                                                                                 |
|-----|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1   | Validation FracGoal 0 for 433k steps; stage flips every rollout           | Random goal from stage 0; 25k-step stage lock and 25% demotion margin               |
| 2   | Stuck at stage 0.33 by wall contacts                                      | Collision thresholds 14/11/9 raised to 22/19/16                                     |
| 3   | Demoted after each promotion as collisions rise                           | Collisions gate promotion only                                                      |
| 4   | Best checkpoint lacks matching normalization; noisy selection; late drift | Save statistics with checkpoint; 30 validation episodes; linear learning-rate decay |

The runs also show how much single runs vary. Runs 2 and 3 differed only in the collision thresholds, which did not bind before 75k steps, yet training FracGoal at 75k was 0.52 in one and 0.30 in the other, because parallel environments are not bit-reproducible. This is why we do not treat the single-seed numbers in Table I as final.

## VII. DISCUSSION

The results support two claims and leave a third open. First, a narrow training distribution was the main cause of the earlier collapse on held-out maps. With procedural layouts, random goals and randomized dynamics, the same algorithm and a similar reward transfer to maps it never saw. Second, the learned policy does better than the hand-written rule when the flock is split by obstacles. The heuristic handles one cluster at a time and rarely finishes before the time limit on that map. We have not yet analyzed the learned trajectories, but the memory features and LSTM state give the policy a way to track sheep it can no longer see, which the heuristic lacks beyond the last seen centroid.

The open question is whether the learned policy can match the heuristic across the whole test distribution. It cannot yet, and three factors are likely involved. The curriculum never trained at full difficulty for long, so the test ranges, which are wider than training at both ends, are far from what the policy mostly saw. The heuristic reads the full obstacle map, while the policy has 24 lidar rays. The reward has no term for pushing a flock through an opening, which is what corridor and narrow-gate maps need.

Several changes follow directly. The demotion bound at stage 0.66 could be relaxed so that most training happens at full breadth. A residual policy on top of the heuristic would keep its open-field competence and let learning handle splits. A reward term or a sub-goal for passing through openings could address the corridor and gate maps. Finally, the clone should be retrained on demonstrations from the procedural distribution before any claim is made about cloning as a method.

*Limitations.* The recurrent PPO numbers come from one seed. The environment is a 2-D kinematic simulation with rectangular obstacles, and sight is not blocked by obstacles. The five presets are single maps and are useful as case studies, not as a distribution. The heuristic's parameters were not retuned for the presets. We control a single herder; cooperation between herders [10], [11] is out of scope.

## VIII. CONCLUSION

We trained a recurrent PPO herder under limited visibility on procedurally generated layouts with an adaptive curriculum, and evaluated it against a collect-and-drive heuristic and a behavioral clone on 150 held-out episodes per scenario. Compared with a policy trained on fixed layouts, success on held-out split-field and open-field maps rose from 0% to 45% and 49%. The learned policy beats the heuristic on the split field, ties it on the dense map, and trails it on the procedural test suite and in open space. Getting the curriculum to work required four fixes: random goals from the first stage, a stage lock with a demotion margin, collision checks used only for promotion, and checkpoints saved with their normalization statistics. The next steps are multi-seed evaluation, longer training at full difficulty, and a reward for moving the flock through openings.

## REFERENCES

- [1] J.-M. Lien, O. B. Bayazit, R. T. Sowell, S. Rodriguez, and N. M. Amato, "Shepherding Behaviors," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2004, pp. 4159–4164. doi: 10.1109/ROBOT.2004.1308924.
- [2] D. Strömbom *et al.*, "Solving the Shepherding Problem: Heuristics for Herding Autonomous, Interacting Agents," *Journal of the Royal Society Interface*, vol. 11, no. 100, p. 20140719, 2014, doi: 10.1098/rsif.2014.0719.
- [3] A. J. King *et al.*, "Biologically Inspired Herding of Animal Groups by Robots," *Methods in Ecology and Evolution*, vol. 14, no. 2, pp. 478–486, 2023, doi: 10.1111/2041-210X.14081.
- [4] R. Vaughan, N. Sumpter, J. Henderson, A. Frost, and S. Cameron, "Experiments in Automatic Flock Control," *Robotics and Autonomous Systems*, vol. 31, no. 1–2, pp. 109–117, 2000, doi: 10.1016/S0921-8890(99)00084-6.
- [5] A. Pierson and M. Schwager, "Controlling Noncooperative Herds with Robotic Herders," *IEEE Transactions on Robotics*, vol. 34, no. 2, pp. 517–525, 2018, doi: 10.1109/TRO.2017.2776308.
- [6] K. Cobbe, O. Klimov, C. Hesse, T. Kim, and J. Schulman, "Quantifying Generalization in Reinforcement Learning," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 1282–1289.
- [7] C. W. Reynolds, "Flocks, Herds and Schools: A Distributed Behavioral Model," in *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1987, pp. 25–34. doi: 10.1145/37401.37406.
- [8] D. Strömbom and A. J. King, "Robot Collection and Transport of Objects: A Biomimetic Process," *Frontiers in Robotics and AI*, vol. 5, p. 48, 2018, doi: 10.3389/frobt.2018.00048.
- [9] A. Hussein, E. Petraki, S. Elsayah, and H. A. Abbass, "Autonomous Swarm Shepherding Using Curriculum-Based Reinforcement Learning," in *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2022, pp. 633–641.
- [10] Y. Hasan, J. E. G. Baxter, C. A. S. Castillo, E. Delgado, and L. Tapia, "Flock Navigation by Coordinated Shepherds via Reinforcement Learning," in *Proceedings of the 15th International Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2022.
- [11] I. Napolitano, A. Lama, F. De Lellis, and M. di Bernardo, "Emergent Cooperative Strategies for Multi-Agent Shepherding via Reinforcement Learning," *arXiv preprint arXiv:2411.05454*, 2024.
- [12] M. Hausknecht and P. Stone, "Deep Recurrent Q-Learning for Partially Observable MDPs," in *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.
- [13] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [14] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. doi: 10.1109/IROS.2017.8202133.
- [15] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey," *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020.
- [16] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 29304–29320.
- [17] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman, Hall/CRC, 1994.
- [18] D. A. Pomerleau, "ALVINN: An Autonomous Land Vehicle in a Neural Network," in *Advances in Neural Information Processing Systems*, 1989.
- [19] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation Learning: A Survey of Learning Methods," *ACM Computing Surveys*, vol. 50, no. 2, pp. 1–35, 2017, doi: 10.1145/3054912.
- [20] S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.
- [21] A. Y. Ng, D. Harada, and S. Russell, "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping," *Proceedings of the 16th International Conference on Machine Learning*, pp. 278–287, 1999.
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [23] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [24] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.